



FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

Towards Fast Automatic Verification of Textbook Proof Steps (Natty + Natural Lean)

Adam Dingle
Charles University, Prague
September 2, 2026

Natty: a natural-language proof assistant

- input: definitions/theorems/proofs in **controlled mathematical English**
- Natty translates proof steps into logical formulas, verifies their correctness
- foundation: **classical higher-order logic** (like HOL Light, Isabelle)
- **internal automatic prover** based on higher-order superposition calculus
- small, experimental system (~6,000 lines of OCaml)
 - but powerful enough to prove hundreds of theorems about $\mathbb{N}/\mathbb{Z}/\mathbb{Q}$

Comparison with Naproche

	Naproche	Natty
input	LaTeX	plain text with Unicode
logic	first-order	higher-order
static type checking	no	yes
proof structure	explicit	implicit
premises for each proof step	explicit	implicit
written in	Haskell	OCaml
IDE	Isabelle	Visual Studio Code
foundation	axioms for $\mathbb{R}$	axioms for $\mathbb{N}$

Natty and Natural Lean

- Natty uses various superposition techniques to prove easy theorems (proof steps) quickly
 - conference paper is about those techniques
- However, I recently decided to end work on Natty and began developing its successor:
Natural Lean
 - $\approx$ rewrite of Natty inside Lean
- **Goals for this talk**
 - summarize superposition techniques in Natty
 - introduce Natural Lean

Natty: sample input text

Theorem (prime divides factor). Let $p, a, b : \mathbb{Z}$. Suppose that p is prime and $p \mid ab$. Then $p \mid a$ or $p \mid b$.

Proof. Suppose that $p \nmid a$. We must show that $p \mid b$. Let $c : \mathbb{Z}$ with $c > 0$, and suppose that $c \mid p$ and $c \mid a$. Since p is prime, $c = p$ or $c = 1$. If $c = p$ then $p \mid a$, which is a contradiction. So $c = 1$. Consequently p and a are relatively prime. Hence by [relatively prime iff linear combination is 1] $1 = xp + ya$ for some $x, y : \mathbb{Z}$. Therefore $1 \cdot b = (xp + ya) \cdot b$, so $b = xbp + yab$. We know that $p \mid ab$, so $p \mid (xb)p + y(ab)$, so $p \mid b$.

- 13 proof steps above, each becomes a **conjecture** (formula) to verify
- Each conjecture has a set of **premises**: known theorems plus facts already established in this proof
- Proof steps may be annotated with **reasons**, but most are not!

Existing provers often fail to verify "easy" proof steps quickly

Theorem (**prime divides factor**). Let $p, a, b : \mathbb{Z}$. Suppose that p is prime and $p \mid ab$. Then $p \mid a$ or $p \mid b$.

Proof. Suppose that $p \nmid a$. We must show that $p \mid b$. Let $c : \mathbb{Z}$ with $c > 0$, and suppose that $c \mid p$ and $c \mid a$. Since p is prime, $c = p$ or $c = 1$. If $c = p$ then $p \mid a$, which is a contradiction. So $c = 1$. Consequently p and a are relatively prime. Hence by [relatively prime iff linear combination is 1] $1 = xp + ya$ for some $x, y : \mathbb{Z}$. Therefore $1 \cdot b = (xp + ya) \cdot b$, so $b = xbp + yab$. We know that $p \mid ab$, so $p \mid (xb)p + y(ab)$, so $p \mid b$.

- Natty can export each conjecture to a **THF file** for testing in external provers
- Vampire (higher-order build, CASC portfolio mode) fails to prove red underlined steps within 10-second time limit

Natty's automatic prover

- Goal: prove "**easy**" steps quickly (ideally in < 1 sec), 100% of the time
 - even when there are many premises to consider!
- Based on pragmatic, incomplete subset of higher-order superposition calculus [1]
- Broadly similar to other saturation-based provers such as E, Vampire
 - Clauses are kept in two sets: **used** (= active) and **unused** (= passive)
 - Initially the unused set contains all premises plus the negated goal
 - At each step: choose a **given clause** from unused set, add it to active set, use **rewriting** to simplify clauses in used set, use **superposition** to generate new clauses
 - Search looks for a proof of $\perp$
- However, some features/heuristics are unusual

Superposition inference rule

$$\frac{D' \vee t \approx t' \quad C\langle u \rangle}{(D' \vee C\langle t' \rangle)\sigma} \quad \text{Sup} \quad (\sigma \text{ unifies } t \text{ and } u)$$

- i. u is not fluid
- ii. u is not a variable
- iii. $t\sigma \not\approx t'\sigma$
- iv. the position of u is eligible in C w.r.t. σ
- v. $t \approx t'$ is maximal in $(D' \vee t \approx t')$ w.r.t. σ
- vi. if $t'\sigma = \perp$, u is in a literal of the form $u \approx \top$
- vii. if $t'\sigma = T$, u is in a literal of the form $u \approx \perp$

- side conditions **reduce the branching factor** while preserving completeness
- even with side conditions, superposition can be **extremely productive**

Set of support strategy

- Natty classifies each premise as
 - an **axiom**, **definition** or **theorem** that appeared earlier in the input file
 - a **hypothesis**: an active assumption, or a fact proven in an earlier proof step
- The negated **goal** is an additional input clause
- We define the **set of support** to be the goal clause plus all hypotheses
- Natty only performs superposition between two clauses if at least one of them is derived from the set of support
- A set of support strategy is complete for first-order resolution if the clauses outside the set of support are consistent (Wos, 1965)
- ... but not for ordered resolution or paramodulation (Bachmair, 1994)
- So this optimization sacrifices completeness for efficiency

Restricting superposition based on proof shape

- Many simple proofs consist of an initial theorem application or definition expansion, followed by a series of simplifying steps
- Natty only considers clause derivations that consist of
 - i. a (possibly empty) series of unrestricted **initial inferences**, followed by
 - ii. a series of **non-increasing inferences**
- An initial inference is either
 - a **definition expansion**
 - a superposition with a theorem that the user has explicitly mentioned as a **reason**
 - a **hypothesis substitution**, i.e. substituting A for B when there is a hypothesis $A = B$
- A superposition is **increasing** if it produces a clause whose weight (after rewriting and simplification) is greater than the weight of any premise in the set of support

Non-destructive rewriting

- Virtually all superposition-based provers include a rewrite rule that simplifies a clause **destructively**
- Rewriting is **efficient** because long chains of rewrites can reduce an expression to a canonical form
- Unfortunately, rewriting a clause may destroy an opportunity for a short proof!
- Natty rewrites some clauses **non-destructively**: the clause is rewritten, but a copy of the original clause is also kept
- If a series of rewrites occurs, Natty keep a copy only of the **original clause**, not of all intermediate clauses in the sequence
- To minimize cost, only used for clauses derived from the last hypothesis or the goal

Commutative unification

- If a premise states that an operation is commutative, most automatic provers will use superposition to produce **commutative variants** of other premises.
- Example:
 - $C_1: \forall a, b: \mathbb{N}. a + b = b + a$ (addition is commutative)
 - $C_2: \forall x: \mathbb{N}. x + 0 = x$
 - $\Rightarrow \forall x: \mathbb{N}. 0 + x = x$ (commutative variant of C_2)
- A large formula may have many commutative variants!
- Natty will not produce commutative variants due to set of support restriction
- Instead, if (+) is known to be commutative then $s + t$ **unifies with** $t + s$
- Commutative unification is NP-complete but seems to be cheap in practice

TextbookMath: an ATP benchmark suite of textbook proofs

- contains several sections of Mendelson, *Number Systems and the Foundations of Analysis* (1973), transcribed as literally as possible into N (Natty's controlled natural language)
- foundation: **Peano axioms** for $\mathbb{N}$, expressed in higher-order logic
 - also defines $\mathbb{Z}$ using $\mathbb{N}$
- **151 theorems** about $\mathbb{N}$, $\mathbb{Z}$
 - addition, multiplication, ordering, division, greatest common divisors, relative primality, prime numbers
- total of **962 proof steps** to verify
 - Natty can generate a THF file for each proof step
- should be a useful test set for ATP systems
 - both monomorphic and polymorphic versions of the library exist
 - E doesn't support polymorphic higher-order logic

Prover performance on TextbookMath

module		Natty + by	Natty	E	Vampire	Zipperposition
div	proved (of 141) avg time	115 (82%) 0.13 sec	113 (80%) 0.14 sec	114 (81%) 0.28 sec	111 (79%) 0.47 sec	101 (72%) 0.69 sec
int	proved (of 483) avg time	456 (94%) 0.05 sec	451 (93%) 0.04 sec	373 (77%) 0.10 sec	390 (81%) 0.50 sec	384 (80%) 0.32 sec
nat	proved (of 301) avg time	299 (99%) 0.02 sec	290 (96%) 0.02 sec	285 (95%) 0.01 sec	280 (93%) 0.16 sec	261 (87%) 0.14 sec
prime	proved (of 37) avg time	23 (62%) 0.07 sec	23 (62%) 0.07 sec	37 (100%) 0.68 sec	37 (100%) 0.08 sec	36 (97%) 1.24 sec
total	proved (of 962) avg time	893 (93%) 0.05 sec	877 (91%) 0.05 sec	809 (84%) 0.12 sec	818 (85%) 0.36 sec	782 (81%) 0.35 sec

- All provers were run with a single default strategy (no portfolio).
- In the **Natty + by** test, Natty was allowed to use **reasons** appearing in proof steps

Natlib: a math library developed in Natty

- foundation: **Peano axioms** for $\mathbb{N}$, expressed in higher-order logic
- develops number systems: $\mathbb{Z}$ defined using $\mathbb{N}$, $\mathbb{Q}$ defined using $\mathbb{Z}$
- **235 theorems** about $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, total of **1597 proof steps** to verify
- proves **5 theorems** from Freek Wiedijk's famous list of 100 theorems
 - e.g. infinitude of primes, irrationality of $\sqrt{2}$
- Natty verifies the entire library in about 75 seconds on my laptop

Prover performance on Natlib

module		Natty + by	Natty	Vampire	Zipperposition
card	proved (of 23) avg time	23 (100%) 0.08 sec	23 (100%) 0.08 sec	14 (61%) 0.71 sec	14 (61%) 0.71 sec
div	proved (of 259) avg time	259 (100%) 0.11 sec	253 (98%) 0.11 sec	172 (66%) 0.63 sec	187 (72%) 0.87 sec
int	proved (of 648) avg time	648(100%) 0.04 sec	637 (98%) 0.04 sec	549 (85%) 0.57 sec	526 (81%) 0.43 sec
nat	proved (of 264) avg time	264 (100%) 0.01 sec	259 (98%) 0.02 sec	240 (91%) 0.17 sec	239 (91%) 0.25 sec
prime	proved (of 130) avg time	130 (100%) 0.11 sec	120 (92%) 0.10 sec	75 (58%) 0.82 sec	38 (29%) 0.54 sec
rat	proved (of 257) avg time	257 (100%) 0.09 sec	252 (98%) 0.08 sec	163 (63%) 1.09 sec	123 (48%) 1.42 sec
real	proved (of 16) avg time	16 (100%) 0.08 sec	14 (88%) 0.08 sec	11 (69%) 0.81 sec	6 (38%) 0.48 sec
total	proved (of 1597) avg time	1597 (100%) 0.06 sec	1558 (98%) 0.06 sec	1224 (77%) 0.59 sec	1133 (71%) 0.58 sec

Weaknesses/limitations of Natty

- No trusted kernel
- Only one tactic (superposition)
- All known theorems are premises for every proof - does not scale
- No quotient types, dependent types
- No way to express computations/algorithms
- Primitive IDE
- I will never convince the global mathematical community to switch from Lean (+ Rocq/Isabelle) to Natty

Introducing Natural Lean

- Goal: allow writing definitions/theorems/proofs in controlled natural language **inside Lean source files**
- Try to stay as close to mathematical textbook English as reasonably possible
- Should be useful both for people writing proofs, and as an autoformalization target
- Demo

Natural Lean: demo points

- Can mix natural and native Lean syntax in the same source file
- The user does not need to specify a tactic for every proof step
 - A **default tactic** is applied automatically
 - Currently applies grind and aesop, but should be configurable
- However, in any proof step the user may provide theorems(s) to be applied, or even an arbitrary tactic
- "f(x)" syntax for function calls (not "f x")
- Easy syntax for equality chains
- Theorems can be grouped

Natural Lean: status and plans

- status
 - about 1000 lines of (dense, functional) Lean code
 - some simple proofs work
 - very limited grammar, poor error messages
 - definitely not ready for real-world use
- plans
 - continue porting Natty grammar into Natural Lean
 - expand to richer grammatical constructs
 - find natural-language formulations of Lean constructs (e.g. type classes, termination)

Natural Lean: challenges

- reasonable error messages
 - especially challenging with hidden layer of translation to Lean
- nice rendering (e.g. variable-width font, symbolic formulas) in Lean's VSCode extension
- Lean has extensible syntax used by many theories, but Natural Lean replaces Lean's syntax - how to unify these?
- Can complex terms with polymorphism and dependent types be expressed nicely in natural language?

Natural Lean: long-term goals

- port Mathlib to Natural Lean, or build a second Mathlib in natural language only
- graphical equation editor

Questions?